

Marabou Resources Directory 탐색

Marabou GitHub 저장소의 resources 디렉터리는 tool paper의 실험 결과 재현에 필요한 모델, 속성 파일, 실행 스크립트를 포함한다. 주요 구성은 다음과 같다.

resources/

- |—— bnn_queries/ # Binary Neural Network 검증 쿼리 예제
- |—— keras/ # Keras(.h5) 형식 모델
- |—— mps/ # MPS 형식 네트워크 (회귀 테스트용)
- |—— nnet/ # .nnet 형식 벤치마크 (ACAS Xu, MNIST 등)
- |—— onnx/ # ONNX 형식 모델 (MNIST, CIFAR-10, Traffic Sign 등)
- |—— properties/ # 검증 속성 파일
- |—— target/ # 검증이 어려운 hard instance
- |—— SplitAndConquerGuide.ipynb
- |—— fashion.ipq
- |—— runMarabou.py

nnet 폴더에는 ACAS Xu, Collision Avoidance, TwinStreams, MNIST용 FC 네트워크가 카테고리별로 정리되어 있다. **onnx** 폴더는 Marabou의 주요 입력 형식으로, MNIST, CIFAR-10, Traffic Sign 등 다양한 도메인의 모델을 제공한다. **properties** 폴더에는 ACAS Xu용 acas_property_1~4.txt와 MNIST용 L-inf robustness 속성 파일이 포함되며, 속성은 입력·출력 변수에 대한 부등식으로 정의된다.

runMarabou.py는 ACAS Xu, MNIST, CIFAR-10 벤치마크를 커맨드라인에서 바로 실행할 수 있는 통합 스크립트로, --dataset 옵션으로 L-inf robustness query를 자동 인코딩한다. **SplitAndConquerGuide.ipynb**는 문제를 서브 문제로 분할해 병렬 처리하는 SnC 모드 사용 가이드다.

지원 모델 형식은 .nnet, .onnx, .pb이며, 속성은 Python API, 속성 파일(.txt), VNNLIB 형식으로 지정할 수 있다.

본 실험에서는 직접 학습한 MNIST 분류용 소형 완전연결 신경망을 사용하였다. 구조는 FC(784→32) + ReLU, FC(32→16) + ReLU, FC(16→10)이며, Adam(lr=1e-3)으로 10 epoch 학습하여 테스트 정확도 약 95%를 달성했다.

이 모델을 선택한 이유는 두 가지다. 첫째, Marabou는 SMT 기반 검증 도구로 ReLU 뉴런 수에 따라 복잡도가 지수적으로 증가하므로, 검증 가능한 크기의 네트워크가 필요했다. 둘째, MNIST는 입력 차원이 작고 digit별 샘플을 쉽게 선택할 수 있어 L-inf robustness query를 명확하게 정의하기 적합하다. 학습된 PyTorch 모델은 ONNX 형식(mnist_fc.onnx)으로 변환하여 Marabou에 로드하였다.

Verification Query 설계

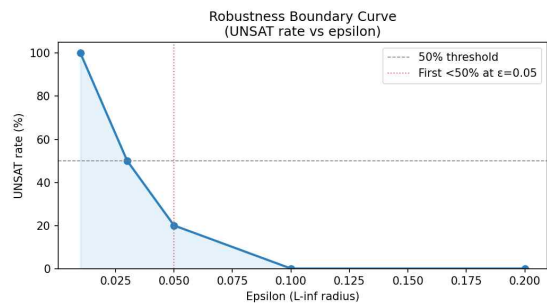
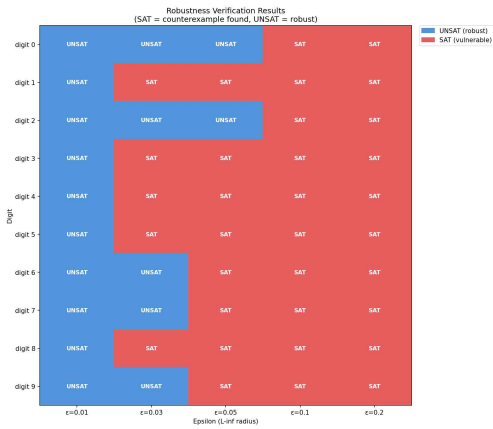
기준 입력 x 가 digit d 로 분류될 때, L-inf ball 안의 모든 x' 도 d 로 분류되는지를 검증하였다. 입력 constraint는 $\max(0, x_i - \epsilon) \leq x'_i \leq \min(1, x_i + \epsilon)$ 로 정의하였다. 출력 constraint는 각 $j \neq d$ 에 대해 $f_d(x') - f_j(x') \leq 0$ 을 만족하는 반례를 탐색

하는 pairwise inequality로 구성하였다. addDisjunctionConstraint의 EquationType 생성자 버그를 우회하기 위해 addInequality 방식을 사용하였으며, 9개 pairwise query가 모두 UNSAT이면 강건성 증명, 하나라도 SAT이면 반례 발견으로 판정하였다.

실험은 digit 0~9 $\times \epsilon \in \{0.01, 0.03, 0.05, 0.1, 0.2\}$, 총 50개 조합으로 수행하였다.

결과 분석

ϵ 값이 증가함에 따라 SAT 비율이 단조 증가하는 명확한 경향을 보였다. $\epsilon=0.01$ 에서는 모든 digit이 UNSAT으로 검증되어 완전한 강건성을 보였으나, $\epsilon=0.03$ 부터 SAT가 발생하기 시작하여 $\epsilon=0.1$ 이후로는 모든 digit에서 반례가 발견되었다.

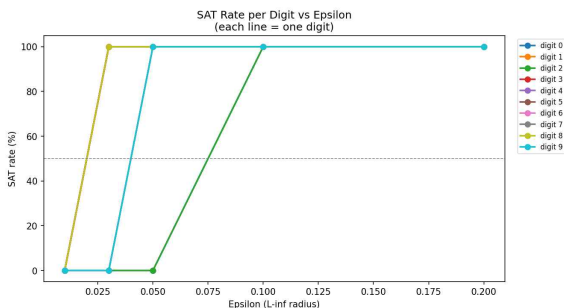


강건성 경계 (Robustness Boundary)

Robustness boundary curve를 보면 UNSAT 비율이 $\epsilon=0.01(100\%) \rightarrow \epsilon=0.03(50\%) \rightarrow \epsilon=0.05(20\%) \rightarrow \epsilon=0.1(0\%)$ 로 급격히 감소한다. 50% 임계선을 처음으로 하회하는 지점은 $\epsilon=0.05$ 이며, 실질적인 강건성 경계 구간은 $\epsilon=0.03 \sim 0.05$ 사이로 볼 수 있다. 이는 이 모델이 픽셀값 기준 약 3~5% 수준의 perturbation부터 취약해지기 시작함을 의미한다.

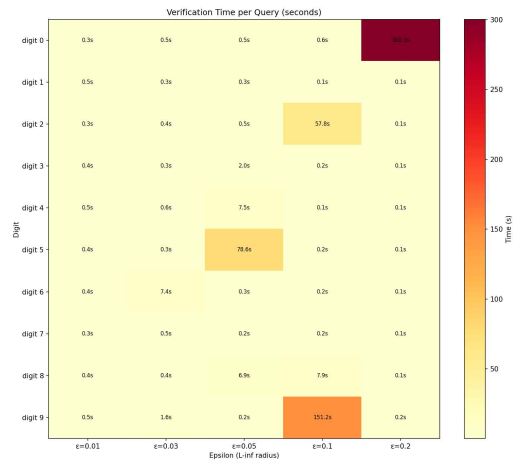
Digit별 취약성

전체 epsilon에 걸친 SAT 비율을 digit별로 보면 digit 0과 2가 40%로 가장 낮아 상대적으로 강건하고, digit 1, 3, 4, 5, 8이 80%로 가장 취약하며, digit 6, 7, 9는 60%로 중간 수준이다. Per-digit SAT rate 그래프에서 이 차이가 더욱 뚜렷하게 나타난다. digit 1, 3, 4, 5, 8은 $\epsilon=0.03$ 에서 이미 SAT로 전환되는 반면, digit 0과 2는 $\epsilon=0.05$ 까지 UNSAT을 유지하다가 $\epsilon=0.1$ 에서 전환된다.



검증 시간

대부분의 쿼리는 1초 미만으로 완료되었으나, 일부 UNSAT 쿼리에서 시간이 크게 증가하였다. 특히 digit 0 / $\epsilon=0.2$ 에서 300.3s(timeout), digit 9 / $\epsilon=0.1$ 에서 151.2s, digit 5 / $\epsilon=0.05$ 에서 78.6s가 소요되었다. $\epsilon=0.1$ 이후 폭발적으로 증가한 시간이 존재한다. 이는 UNSAT 쿼리가 전체 perturbation 공간을 탐색해야 하기 때문이다. SAT 쿼리는 반례 발견 즉시 종료되므로 빠르게 끝나는 반면, UNSAT 쿼리는 exhaustive search가 필요하다.



Marabou 강점

첫째, UNSAT 결과는 exhaustive proof다. 일반적인 adversarial attack 기법은 반례를 찾지 못하더라도 강건성을 보장하지 못하지만, Marabou의 UNSAT은 주어진 constraint 범위 안에서 반례가 존재하지 않음을 수학적으로 증명한다.

둘째, SAT 시 구체적인 반례를 반환한다. 어떤 perturbation이 오분류를 유발하는지 정확한 입력값을 확인할 수 있어 모델 분석 및 디버깅에 활용될 수 있다.

셋째, pip를 통해 간편하게 설치할 수 있으며, ONNX 지원과 Python API(maraboupy)를 통해 PyTorch 학습 워크플로우와 자연스럽게 통합된다. 별도의 복잡한 변환 작업 없이 표준 export 파이프라인으로 연결이 가능하다.

Marabou 한계

첫째, 확장성 문제가 가장 큰 한계다. ReLU 뉴런 수에 따라 검증 복잡도가 지수적으로 증가한다. 본 실험에서 사용한 $784 \rightarrow 32 \rightarrow 16 \rightarrow 10$ 의 소형 네트워크에서도 일부 쿼리가 300s timeout에 도달했으며, 더 깊은 CNN이나 대규모 모델에는 현실적으로 적용이 어렵다.

둘째, UNSAT 쿼리의 검증 시간 변동성이 크다. 동일한 모델과 동일한 ϵ 를 사용해도 digit과 입력 위치에 따라 runtime이 수백 배 차이가 났다. 이는 검증 계획 수립을 어렵게 만든다.